

EM - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4298 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4021 values"
## [1] "277 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	519	33	107	NA	NA	659
2	60	NA	400	80	101	641
3	273	NA	155	194	15	637
4	275	NA	217	64	36	592
5	227	NA	187	69	35	518
6	198	1	124	66	22	411
7	168	NA	98	27	18	311
8	89	NA	52	25	4	170
9	73	NA	2	NA	7	82

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7875569	0.0500759	0.1623672	NA	NA	659
2	0.0936037	NA	0.6240250	0.1248050	0.1575663	641
3	0.4285714	NA	0.2433281	0.3045526	0.0235479	637
4	0.4645270	NA	0.3665541	0.1081081	0.0608108	592
5	0.4382239	NA	0.3610039	0.1332046	0.0675676	518
6	0.4817518	0.0024331	0.3017032	0.1605839	0.0535280	411
7	0.5401929	NA	0.3151125	0.0868167	0.0578778	311
8	0.5235294	NA	0.3058824	0.1470588	0.0235294	170
9	0.8902439	NA	0.0243902	NA	0.0853659	82

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

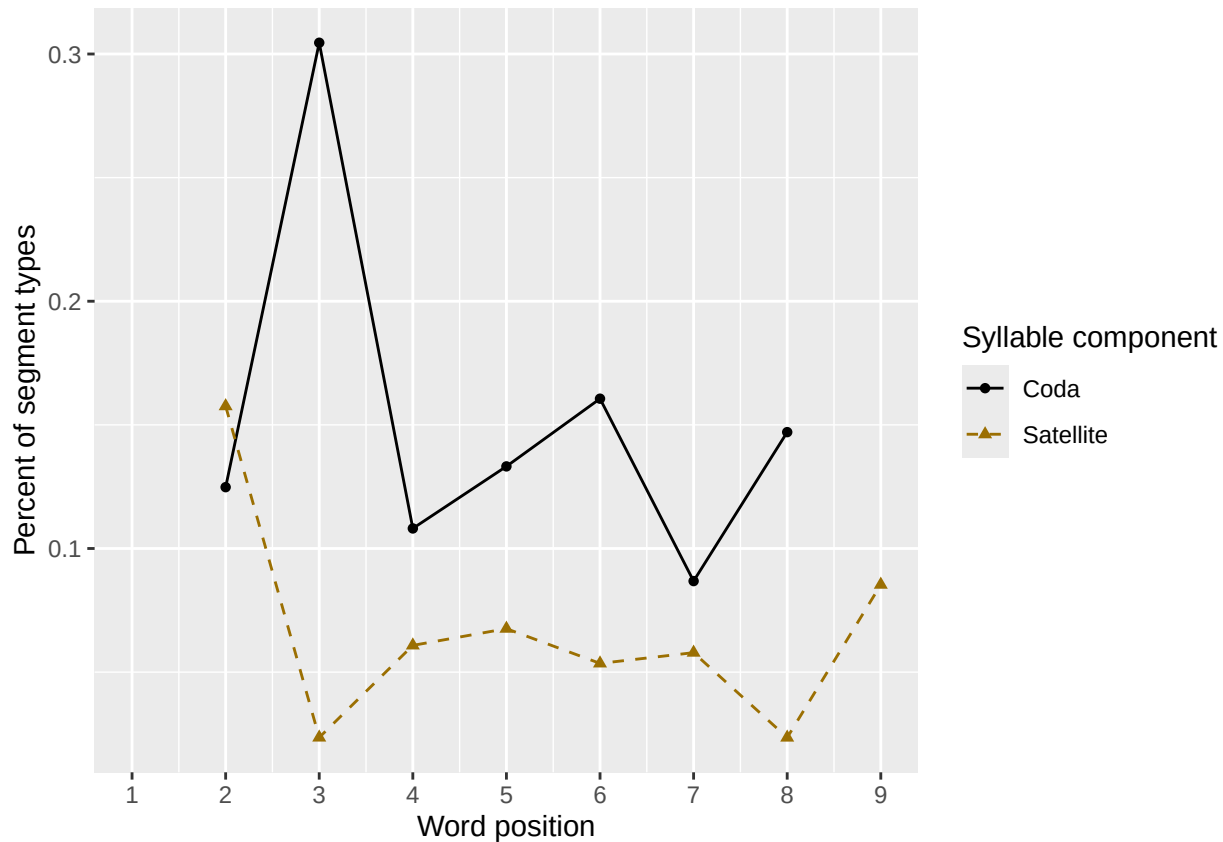
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.623 0.868 0.774 NA      NA      NA      NA      NA      NA
## 2      5 0.482 0.756 0.563 0.540 NA      NA      NA      NA      NA
## 3      6 0.436 0.667 0.594 0.76  0.660 NA      NA      NA      NA
## 4      7 0.422 0.631 0.481 0.544 0.792 0.787 NA      NA      NA
## 5      8 0.477 0.7   0.557 0.664 0.604 0.817 0.735 NA      NA
## 6      9 0.310 0.561 0.481 0.65  0.512 0.536 0.826 0.730 NA
## 7     10 0.388 0.641 0.553 0.618 0.641 0.571 0.679 0.877 0.829

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    53    53    53    NA    NA    NA    NA    NA    NA
## 2      5    85    86    87    87    NA    NA    NA    NA    NA
## 3      6   101    99    96   100   106    NA    NA    NA    NA
## 4      7   102   103   104   103   106   108    NA    NA    NA
## 5      8   151   140   140   146   144   142   147    NA    NA
## 6      9    87    82    81    80    84    84    86    89    NA
## 7     10    80    78    76    76    78    77    78    81    82

```

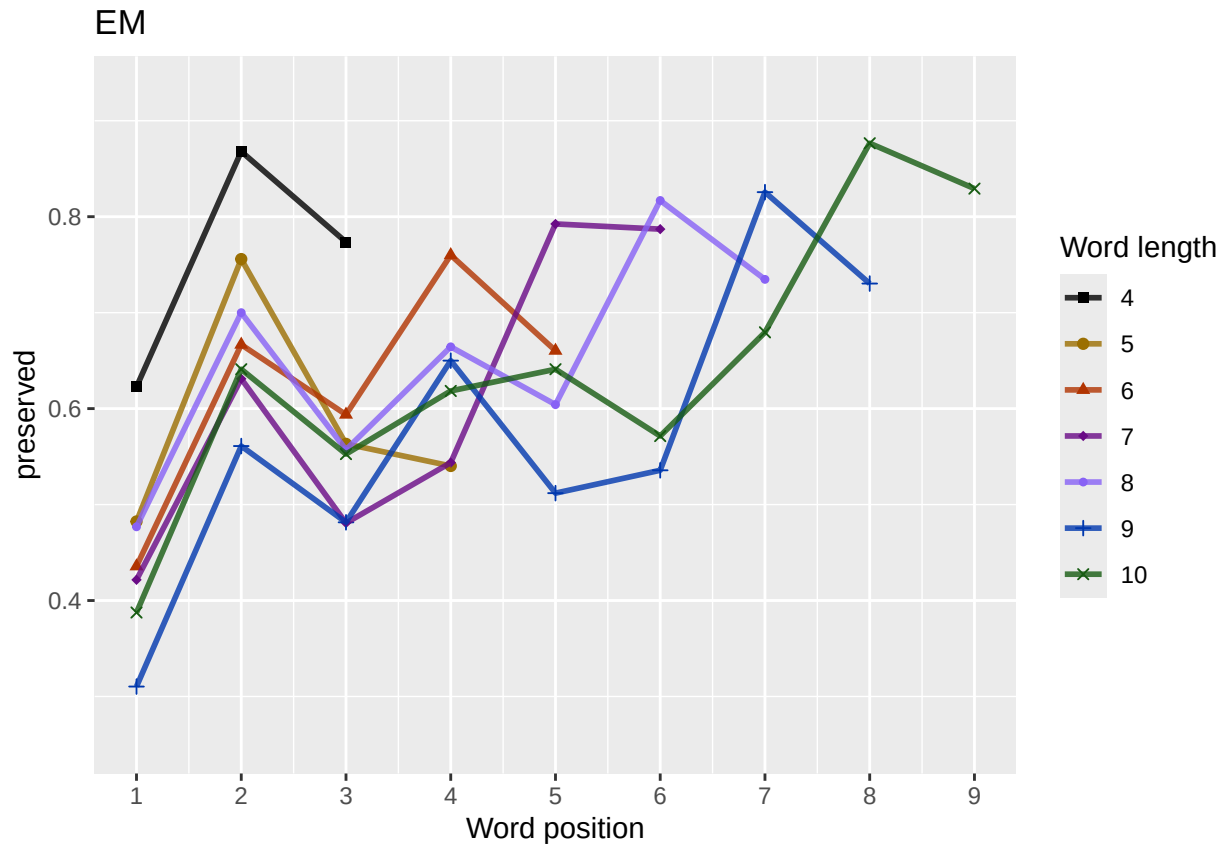
```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 4
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen      pos
##      0.5945      -0.1106      0.2051
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5165  AIC: 5171
## log likelihood:  -2582.273
## Nagelkerke R2:  0.05017185
## % pres/err predicted correctly:  -1811.78
## % of predictable range [ (model-null)/(1-null) ]:  0.03698178
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos
##      0.85581      -0.14258      0.11605      0.01056
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5164  AIC: 5172
## log likelihood:  -2581.803
## Nagelkerke R2:  0.05047837
## % pres/err predicted correctly:  -1811.485
## % of predictable range [ (model-null)/(1-null) ]:  0.03713862
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      0.680861      -0.113036      0.005417      0.160372
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5164  AIC: 5172
## log likelihood:  -2581.995
## Nagelkerke R2:  0.05035336
## % pres/err predicted correctly:  -1811.804
## % of predictable range [ (model-null)/(1-null) ]:  0.03696914
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)  stimlen:pos

```

```

##          0.431967          -0.089592          -0.042487          0.423102          0.005009          -0.0
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4015 Residual
## Null Deviance:          5315
## Residual Deviance: 5163 AIC: 5175
## log likelihood:  -2581.321
## Nagelkerke R2:  0.05079318
## % pres/err predicted correctly:  -1811.184
## % of predictable range [ (model-null)/(1-null) ]:  0.03729831
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##   -0.1332         0.1727
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:          5315
## Residual Deviance: 5192 AIC: 5196
## log likelihood:  -2596.069
## Nagelkerke R2:  0.04112834
## % pres/err predicted correctly:  -1825.117
## % of predictable range [ (model-null)/(1-null) ]:  0.02989662
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##  -0.1431877   -0.0007766    0.1791957
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:          5315
## Residual Deviance: 5192 AIC: 5198
## log likelihood:  -2596.063
## Nagelkerke R2:  0.04113219
## % pres/err predicted correctly:  -1825.067
## % of predictable range [ (model-null)/(1-null) ]:  0.02992306
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)
##    0.516
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4020 Residual

```



```
## Null Deviance:          5315
## Residual Deviance: 5315 AIC: 5317
## log likelihood: -2657.643
## Nagelkerke R2: 0
## % pres/err predicted correctly: -1881.394
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      0.61277      -0.01256
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance:          5315
## Residual Deviance: 5315 AIC: 5319
## log likelihood: -2657.432
## Nagelkerke R2: 0.0001427821
## % pres/err predicted correctly: -1881.199
## % of predictable range [ (model-null)/(1-null) ]: 0.0001038031
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~	5170.545	0.000000	1.000000	0.453968	0.105017	0.85944905	-	0.2050816	NA	NA
stimlen + pos						0.1105837				
preserved ~	5171.607	0.061471	0.588172	0.226701	0.0450478	0.8558108	-	0.1160490	0.105606	NA
stimlen * pos						0.1425790				
preserved ~	5171.989	0.444265	0.485715	0.322049	0.0250359	0.46808615	-	0.1603721	NA	0.0054174
stimlen + I(pos^2)						0.1130356				
+ pos										
preserved ~	5174.642	0.097332	0.128906	0.0705851	0.050799	0.24319673	-	0.4231018	-	0.0050095
stimlen * (I(pos^2)						0.0895923			0.0266908	0.0424867
+ pos)										

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	5196.1375	591956	0.000000	0.000000	0.30411283	-	NA	0.1726978	NA	NA	NA
						0.1331636					
preserved ~ I(pos^2) + pos	5198.1237	580235	0.000000	0.000000	0.50411322	-	NA	0.1791957	NA	-	NA
						0.1431877				0.0007766	
preserved ~ 1	5317.2854	674068	0.000000	0.000000	0.000000	0.05160018	NA	NA	NA	NA	NA
preserved ~ stimlen	5318.8654	831962	0.000000	0.000000	0.00001428	6127737	-	NA	NA	NA	NA
						0.0125643					

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      0.5945      -0.1106       0.2051
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5165 AIC: 5171
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.588 0.637 0.683 NA     NA     NA     NA     NA     NA
## 2     5 0.561 0.611 0.659 0.703 NA     NA     NA     NA     NA
## 3     6 0.534 0.584 0.633 0.679 0.722 NA     NA     NA     NA
## 4     7 0.506 0.557 0.607 0.655 0.700 0.741 NA     NA     NA
## 5     8 0.479 0.530 0.581 0.630 0.676 0.719 0.759 NA     NA
## 6     9 0.451 0.502 0.553 0.603 0.651 0.696 0.738 0.776 NA
## 7    10 0.424 0.475 0.526 0.577 0.626 0.672 0.716 0.756 0.792
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

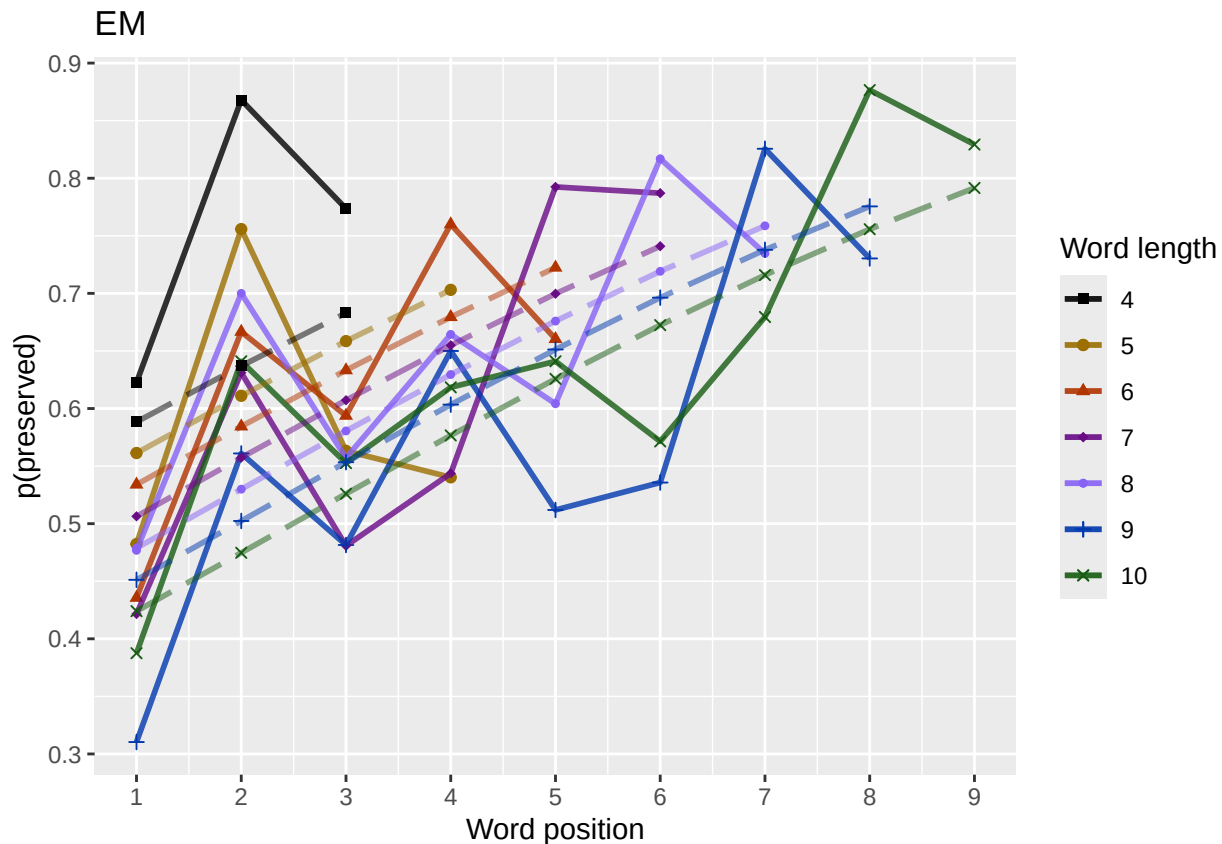
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      4    691

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 4 / 691 = 0.58 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      0.5728      -0.1092       0.2105
##
## Degrees of Freedom: 4009 Total (i.e. Null); 4007 Residual
## Null Deviance:      5294
## Residual Deviance: 5136 AIC: 5142
## log likelihood: -2568.199
## Nagelkerke R2: 0.05245035
## % pres/err predicted correctly: -1800.407
## % of predictable range [ (model-null)/(1-null) ]: 0.03860537
## *****

```

```

## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    0.88952    -0.14799    0.10219    0.01286
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4006 Residual
## Null Deviance:      5294
## Residual Deviance: 5135 AIC: 5143
## log likelihood: -2567.515
## Nagelkerke R2:  0.05289838
## % pres/err predicted correctly: -1799.972
## % of predictable range [ (model-null)/(1-null) ]:  0.0388374
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    0.678210    -0.112148    0.006664    0.155626
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4006 Residual
## Null Deviance:      5294
## Residual Deviance: 5136 AIC: 5144
## log likelihood: -2567.787
## Nagelkerke R2:  0.05272056
## % pres/err predicted correctly: -1800.398
## % of predictable range [ (model-null)/(1-null) ]:  0.03861004
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##    0.392060    -0.085440    -0.049824    0.462482    0.005908    -0.005908
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4004 Residual
## Null Deviance:      5294
## Residual Deviance: 5134 AIC: 5146
## log likelihood: -2566.851
## Nagelkerke R2:  0.0533325
## % pres/err predicted correctly: -1799.583
## % of predictable range [ (model-null)/(1-null) ]:  0.03904519
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      -0.1461      0.1787
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4008 Residual
## Null Deviance:      5294
## Residual Deviance: 5163 AIC: 5167
## log likelihood:  -2581.6
## Nagelkerke R2:  0.0436522
## % pres/err predicted correctly:  -1813.397
## % of predictable range [ (model-null)/(1-null) ]:  0.03167301
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##  -0.1394544   0.0005184   0.1743478
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4007 Residual
## Null Deviance:      5294
## Residual Deviance: 5163 AIC: 5169
## log likelihood:  -2581.597
## Nagelkerke R2:  0.04365389
## % pres/err predicted correctly:  -1813.426
## % of predictable range [ (model-null)/(1-null) ]:  0.03165721
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.5233
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4009 Residual
## Null Deviance:      5294
## Residual Deviance: 5294 AIC: 5296
## log likelihood:  -2646.793
## Nagelkerke R2:  3.029711e-16
## % pres/err predicted correctly:  -1872.744
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)      stimlen
##      0.596289    -0.009476
##
## Degrees of Freedom: 4009 Total (i.e. Null);  4008 Residual
## Null Deviance:      5294
## Residual Deviance: 5293  AIC: 5297
## log likelihood:  -2646.674
## Nagelkerke R2:   8.108886e-05
## % pres/err predicted correctly:  -1872.633
## % of predictable range [ (model-null)/(1-null) ]:  5.888722e-05
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~	5142.399	0.0000000	0.0000000	0.0000000	0.4036522	0.5245045	0.727968	-	0.2104617	NA	NA
stimlen + pos							0.1091790				
preserved ~	5143.029	0.6304380	0.7296290	0.2945164	0.5289848	0.895192	-	0.1021878	0.128597	NA	NA
stimlen * pos							0.1479946				
preserved ~	5143.578	1.1740558	0.5559703	0.2244205	0.5052720	0.6782099	-	0.1556261	NA	0.0066639	NA
stimlen + I(pos^2)							0.1121480				
+ pos											
preserved ~	5145.702	3.3029360	0.1917682	0.0774076	0.5333253	0.3920601	-	0.4624821	-	-	0.0059076
stimlen * (I(pos^2)							0.0854398		0.0311070	0.0498244	
+ pos)											
preserved ~ pos	5167.202	24.8007174	0.0000000	0.0000000	0.0704365	0.22	-	NA	0.1786715	NA	NA
							0.1461046				
preserved ~	5169.192	6.7955903	0.0000000	0.0000000	0.0604365	0.39	-	NA	0.1743478	NA	0.0005184
I(pos^2) + pos							0.1394544				
preserved ~ 1	5295.585	3.1867041	0.0000000	0.0000000	0.0000000	0.5233474	NA	NA	NA	NA	NA
preserved ~	5297.347	54.9484235	0.0000000	0.0000000	0.0000000	0.1596288	-	NA	NA	NA	NA
stimlen							0.0094757				

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

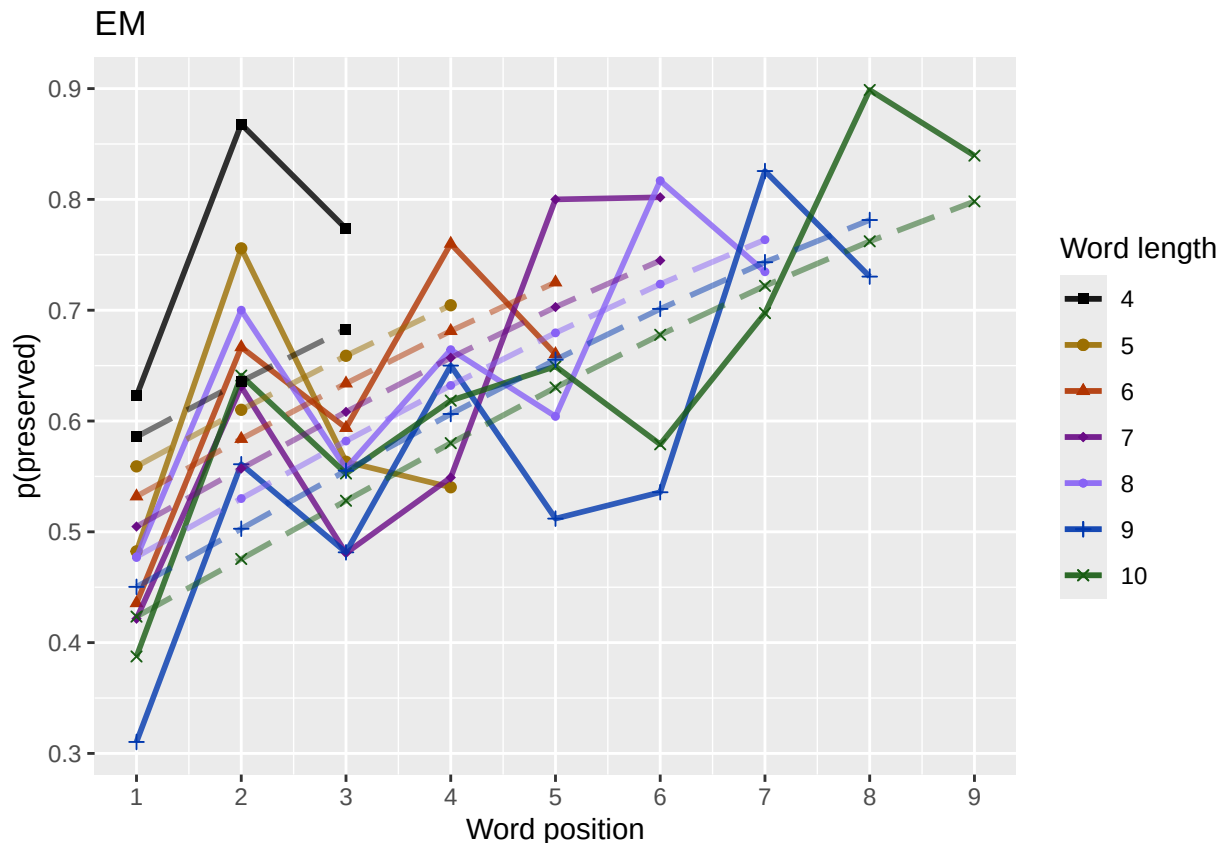
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.586 0.636 0.683 NA      NA      NA      NA      NA      NA
## 2      5 0.559 0.610 0.659 0.704 NA      NA      NA      NA      NA
## 3      6 0.532 0.584 0.634 0.681 0.725 NA      NA      NA      NA
## 4      7 0.505 0.557 0.608 0.657 0.703 0.745 NA      NA      NA
## 5      8 0.477 0.530 0.582 0.632 0.680 0.724 0.764 NA      NA
## 6      9 0.450 0.503 0.555 0.606 0.655 0.701 0.743 0.781 NA
## 7     10 0.423 0.476 0.528 0.580 0.630 0.678 0.722 0.762 0.798
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("no_fragments_percent_preserved_by_length_pos_wfit.png"))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.25 - 0.93"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.0254691
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] 0.04426083
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

CurrentLabel<-"Average upward change after U minimum"
print(CurrentLabel)
print(OA_mean_pos_u_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

CurrentLabel<-"Proportion of average downward change"
prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
}
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      log_freq      pos  stimlen:log_freq
##      0.41318      -0.08992      0.23166      0.20546      -0.02422
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4016 Residual
## Null Deviance:      5315
## Residual Deviance: 5153 AIC: 5163
## log likelihood: -2576.613
## Nagelkerke R2:  0.05386402
## % pres/err predicted correctly: -1806.468
## % of predictable range [ (model-null)/(1-null) ]:  0.03980387
## *****
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stimlen:log
##      0.485830      -0.092070      0.229236      0.004482      0.168476      -0.0
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4015 Residual
## Null Deviance:      5315
## Residual Deviance: 5153 AIC: 5165
## log likelihood: -2576.424
## Nagelkerke R2:  0.0539869
## % pres/err predicted correctly: -1806.524
## % of predictable range [ (model-null)/(1-null) ]:  0.03977395
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      pos  stimlen:log_freq      log_fr
##      0.4130946      -0.0898267      0.2318276      0.2052800      -0.0239150      -0.0
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4015 Residual
## Null Deviance:      5315
## Residual Deviance: 5153 AIC: 5165
## log likelihood: -2576.61
## Nagelkerke R2:  0.05386592
## % pres/err predicted correctly: -1806.465
## % of predictable range [ (model-null)/(1-null) ]:  0.03980533
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos      log_freq
##      0.48366      -0.09622      0.20536      0.04423

```

```

##
## Degrees of Freedom: 4020 Total (i.e. Null); 4017 Residual
## Null Deviance: 5315
## Residual Deviance: 5159 AIC: 5167
## log likelihood: -2579.288
## Nagelkerke R2: 0.05212014
## % pres/err predicted correctly: -1808.988
## % of predictable range [ (model-null)/(1-null) ]: 0.03846504
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 0.47364 -0.09441 0.20337 0.07222 -0.00759
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4016 Residual
## Null Deviance: 5315
## Residual Deviance: 5158 AIC: 5168
## log likelihood: -2578.88
## Nagelkerke R2: 0.05238612
## % pres/err predicted correctly: -1808.598
## % of predictable range [ (model-null)/(1-null) ]: 0.03867225
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 0.567827 -0.098633 0.005265 0.161912 0.044112
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4016 Residual
## Null Deviance: 5315
## Residual Deviance: 5158 AIC: 5168
## log likelihood: -2579.026
## Nagelkerke R2: 0.0522909
## % pres/err predicted correctly: -1809.018
## % of predictable range [ (model-null)/(1-null) ]: 0.03844889
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 0.4860668 -0.0920903 0.2338077 0.0045325 0.1682009
## log_freq:pos
## -0.0024034
##

```

```

## Degrees of Freedom: 4020 Total (i.e. Null); 4013 Residual
## Null Deviance: 5315
## Residual Deviance: 5153 AIC: 5169
## log likelihood: -2576.421
## Nagelkerke R2: 0.05398872
## % pres/err predicted correctly: -1806.525
## % of predictable range [ (model-null)/(1-null) ]: 0.03977331
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos
## 0.5945 -0.1106 0.2051
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance: 5315
## Residual Deviance: 5165 AIC: 5171
## log likelihood: -2582.273
## Nagelkerke R2: 0.05017185
## % pres/err predicted correctly: -1811.78
## % of predictable range [ (model-null)/(1-null) ]: 0.03698178
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 0.5435095 -0.0964239 0.0042614 0.1678664 0.0571493
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4014 Residual
## Null Deviance: 5315
## Residual Deviance: 5157 AIC: 5171
## log likelihood: -2578.664
## Nagelkerke R2: 0.05252682
## % pres/err predicted correctly: -1808.619
## % of predictable range [ (model-null)/(1-null) ]: 0.03866125
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 0.85581 -0.14258 0.11605 0.01056
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4017 Residual
## Null Deviance: 5315
## Residual Deviance: 5164 AIC: 5172

```

```

## log likelihood: -2581.803
## Nagelkerke R2: 0.05047837
## % pres/err predicted correctly: -1811.485
## % of predictable range [ (model-null)/(1-null) ]: 0.03713862
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    0.680861    -0.113036    0.005417    0.160372
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5164 AIC: 5172
## log likelihood: -2581.995
## Nagelkerke R2: 0.05035336
## % pres/err predicted correctly: -1811.804
## % of predictable range [ (model-null)/(1-null) ]: 0.03696914
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stimlen:I(pos^2)
##      0.431967      -0.089592      -0.042487      0.423102      0.005009      -0.000000
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4015 Residual
## Null Deviance:      5315
## Residual Deviance: 5163 AIC: 5175
## log likelihood: -2581.321
## Nagelkerke R2: 0.05079318
## % pres/err predicted correctly: -1811.184
## % of predictable range [ (model-null)/(1-null) ]: 0.03729831
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##   -0.15791      0.17954      0.06561
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5178 AIC: 5184
## log likelihood: -2588.944
## Nagelkerke R2: 0.04580622
## % pres/err predicted correctly: -1818.281

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.03352833
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##    -0.15464      0.17727      0.10505     -0.01088
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5176 AIC: 5184
## log likelihood: -2588.103
## Nagelkerke R2: 0.04635717
## % pres/err predicted correctly: -1817.476
## % of predictable range [ (model-null)/(1-null) ]: 0.03395589
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      log_freq I(pos^2):log_freq
##    -0.168100     -0.001231      0.186776      0.085232     -0.001571
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4015 Residual
## Null Deviance:      5315
## Residual Deviance: 5176 AIC: 5188
## log likelihood: -2588.008
## Nagelkerke R2: 0.04641945
## % pres/err predicted correctly: -1817.321
## % of predictable range [ (model-null)/(1-null) ]: 0.03403811
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##    -0.1332      0.1727
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5192 AIC: 5196
## log likelihood: -2596.069
## Nagelkerke R2: 0.04112834
## % pres/err predicted correctly: -1825.117
## % of predictable range [ (model-null)/(1-null) ]: 0.02989662
## *****
## model index: 19

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
## -0.1431877   -0.0007766    0.1791957
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5192 AIC: 5198
## log likelihood: -2596.063
## Nagelkerke R2:  0.04113219
## % pres/err predicted correctly: -1825.067
## % of predictable range [ (model-null)/(1-null) ]:  0.02992306
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq stimlen:log_freq
##  0.437156      0.007611      0.225757      -0.023563
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5304 AIC: 5312
## log likelihood: -2651.912
## Nagelkerke R2:  0.003881303
## % pres/err predicted correctly: -1876.079
## % of predictable range [ (model-null)/(1-null) ]:  0.002823634
## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##  0.50588      0.00142      0.04278
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5309 AIC: 5315
## log likelihood: -2654.53
## Nagelkerke R2:  0.00210939
## % pres/err predicted correctly: -1878.478
## % of predictable range [ (model-null)/(1-null) ]:  0.001549569
## *****
## model index:  14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```

##
## Coefficients:
## (Intercept)
##      0.516
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4020 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5317
## log likelihood:  -2657.643
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1881.394
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      0.61277      -0.01256
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5319
## log likelihood:  -2657.432
## Nagelkerke R2:  0.0001427821
## % pres/err predicted correctly:  -1881.199
## % of predictable range [ (model-null)/(1-null) ]:  0.0001038031
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)

```

Model	AIC Delta	AIC	AICw	NagR	R ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	len:I(pos^2)
preserved ~ stimlen * log_freq + pos	5163.226	0.000000	0.000000	0.955	0.238641	1828	0.2316602	0.2052578	NA	NA	NA	NA	NA
							0.0899199	0.0242214					
preserved ~ stimlen * log_freq + I(pos^2) + pos	5164.842	2.742140	0.43708	0.85398	0.1858298		0.2292365	0.1688762	NA	0.0044819	NA	NA	NA
							0.0920697	0.0239296					
preserved ~ stimlen * log_freq + pos *	5165.129	4.036895	0.58035	0.86513	0.0946		0.2318276	0.2052800	-	NA	NA	NA	NA
							0.0898267	0.0239150		0.0006886			
log_freq preserved ~ stimlen + pos + log_freq	5166.375	0.000872	0.80042	0.21836	0.6649	0.0442287	0.2052578	NA	NA	NA	NA	NA	NA
							0.0962197						
preserved ~ stimlen + pos * log_freq	5167.453	4.820036	0.70610	0.92387	0.6399	0.0722241	0.2033696	NA	NA	NA	NA	NA	NA
							0.0944126	0.0075896					
preserved ~ stimlen + I(pos^2) + pos + log_freq	5168.452	6.827896	0.93844	0.82290	0.8269	0.0441124	0.1619124	NA	0.0052651	NA	NA	NA	NA
							0.0986326						
preserved ~ stimlen * log_freq + (I(pos^2) + pos) *	5168.543	7.050602	0.95695	0.83857	0.6668	0.2338077	0.1682009	-	0.0045325	0.0002871	NA	NA	NA
							0.0920903	0.0240431		0.0024034			
log_freq preserved ~ stimlen + pos	5170.743	1.956125	0.38110	0.59075	0.44905	NA	NA	0.2052578	NA	NA	NA	NA	NA
							0.1105837						
preserved ~ stimlen + (I(pos^2) + pos) *	5171.820	3.822176	0.85747	0.52626	0.5095	0.057193	0.1678661	0.0042614	NA	NA	NA	NA	NA
							0.0964239			0.0009708			
log_freq preserved ~ stimlen * pos	5171.803	1.038150	0.38660	0.57085	0.58108	NA	NA	0.1160496	NA	NA	NA	0.0105306	NA
							0.1425790						
preserved ~ stimlen + I(pos^2) + pos	5171.956	3.833125	0.05670	0.03538	0.8615	NA	NA	0.1603721	NA	0.0054174	NA	NA	NA
							0.1130356						
preserved ~ stimlen * (I(pos^2) + pos)	5174.512	1.689930	0.78012	0.75093	0.19673	NA	NA	0.4231118	NA	-	NA	NA	-
							0.0895923			0.0424867		0.0266908	0.0050095
preserved ~ pos + log_freq	5183.288	6.282700	0.32601	0.14580	0.62	NA	0.0656141	0.1795446	NA	NA	NA	NA	NA
							0.1579088						

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	stimlen	log_freq	pos	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)	len:I(pos^2)
preserved ~ pos *	5184.207	9803.100	0.278	0.146	3572 NA	0.1050148	0.1772662 NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq					0.1546395		0.0108846								
preserved ~ (I(pos^2) + pos) *	5188.217	9910.000	0.000	0.146	4195 NA	0.0852346	0.1867762 NA	-	-	NA	NA	NA	NA	NA	NA
log_freq					0.1681000					0.0012308	1.5711				
preserved ~ pos	5196.327	10523.000	0.000	0.112	83 NA	NA	NA	0.1728978 NA	NA	NA	NA	NA	NA	NA	NA
log_freq					0.1331636										
preserved ~ I(pos^2) + pos	5198.345	9980.200	0.000	0.000	11322 NA	NA	NA	0.1791957 NA	-	NA	NA	NA	NA	NA	NA
log_freq					0.1431877					0.0007766					
preserved ~ stimlen *	5311.828	5984.800	0.000	0.000	388137	0.5047012	57568 NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq							0.0235629								
preserved ~ stimlen + log_freq	5315.151	6891.200	0.000	0.000	203905	88401020	427740 NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ 1	5317.256	6002.300	0.000	0.000	31600	0.18 NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ stimlen	5318.865	6991.800	0.000	0.000	0.428	27737 NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq							0.0125643								

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + pos"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          pos  stimlen:log_freq
##      0.41318       -0.08992         0.23166       0.20546       -0.02422
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4016 Residual
## Null Deviance:      5315
## Residual Deviance: 5153  AIC: 5163
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq < median_freq]<-"lf"
```

```
PosDat$FLPFitted<-fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

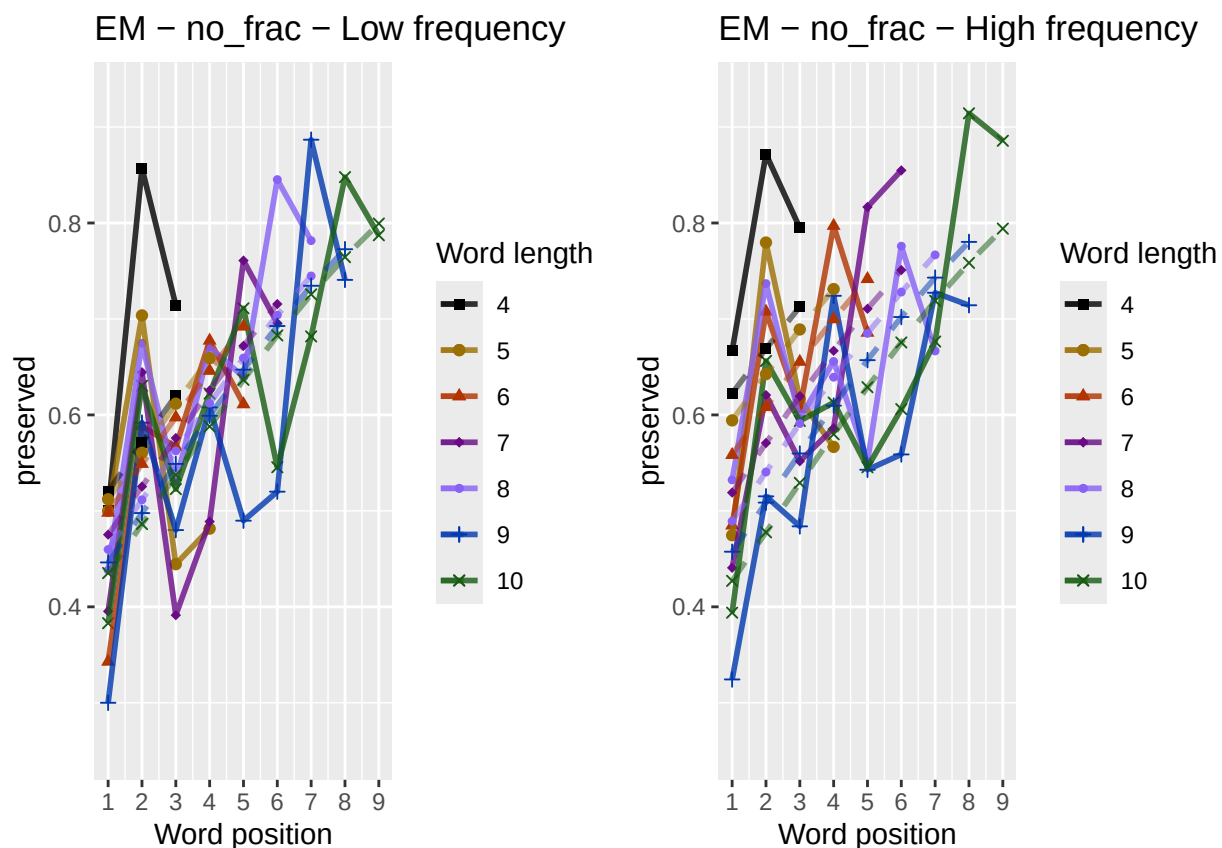
```
HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
             CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations, PosDat)
```

```
## *****
```

```

## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      0.0657      0.3156
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5129 AIC: 5133
## log likelihood: -2564.338
## Nagelkerke R2: 0.06183612
## % pres/err predicted correctly: -1799.382
## % of predictable range [ (model-null)/(1-null) ]: 0.04356838
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      -0.1332      0.1727
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5192 AIC: 5196
## log likelihood: -2596.069
## Nagelkerke R2: 0.04112834
## % pres/err predicted correctly: -1825.117
## % of predictable range [ (model-null)/(1-null) ]: 0.02989662
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      -0.1431877      -0.0007766      0.1791957
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5192 AIC: 5198
## log likelihood: -2596.063
## Nagelkerke R2: 0.04113219
## % pres/err predicted correctly: -1825.067
## % of predictable range [ (model-null)/(1-null) ]: 0.02992306
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      0.3827      0.1166
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5298  AIC: 5302
## log likelihood:  -2649.074
## Nagelkerke R2:  0.005799266
## % pres/err predicted correctly:  -1873.422
## % of predictable range [ (model-null)/(1-null) ]:  0.004235471
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.516
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4020 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5317
## log likelihood:  -2657.643
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1881.394
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      0.61277      -0.01256
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5319
## log likelihood:  -2657.432
## Nagelkerke R2:  0.0001427821
## % pres/err predicted correctly:  -1881.199
## % of predictable range [ (model-null)/(1-null) ]:  0.0001038031
## *****
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)

```



```

MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	5132.670	0.00000	1	1	0.061836	0.065695	30.3156066	NA	NA	NA	NA
preserved ~ pos	5196.137	63.46181	0	0	0.0411283	-	NA	NA	NA	0.1726978	NA
					0.1331636						
preserved ~ (I(pos^2) + pos)	5198.125	5.45009	0	0	0.0411322	-	NA	NA	-	0.1791957	NA
					0.1431877				0.0007766		
preserved ~ CumErr	5302.148	169.47281	0	0	0.0057990	0.3826782	NA	0.1165928	NA	NA	NA
preserved ~ 1	5317.281	184.61055	0	0	0.0000000	0.5160018	NA	NA	NA	NA	NA
preserved ~ stimlen	5318.865	186.18948	0	0	0.0001428	0.6127737	NA	NA	NA	NA	-
											0.0125643

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
}

```

```

BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.4438095	525
O	0.5409139	1882
P	0.0294118	34
S	0.3319328	238
V	0.8845007	1342

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      0.1849      0.3124
##
## Degrees of Freedom: 3748 Total (i.e. Null); 3747 Residual
## Null Deviance:      4853
## Residual Deviance: 4693 AIC: 4697
## log likelihood: -2346.437
## Nagelkerke R2: 0.05764063
## % pres/err predicted correctly: -1636.063

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.03990471
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## -0.01178      0.16771
##
## Degrees of Freedom: 3748 Total (i.e. Null); 3747 Residual
## Null Deviance: 4853
## Residual Deviance: 4748 AIC: 4752
## log likelihood: -2373.812
## Nagelkerke R2: 0.03822487
## % pres/err predicted correctly: -1657.322
## % of predictable range [ (model-null)/(1-null) ]: 0.02743683
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
## 0.005049      0.001315      0.156748
##
## Degrees of Freedom: 3748 Total (i.e. Null); 3746 Residual
## Null Deviance: 4853
## Residual Deviance: 4748 AIC: 4754
## log likelihood: -2373.797
## Nagelkerke R2: 0.03823581
## % pres/err predicted correctly: -1657.387
## % of predictable range [ (model-null)/(1-null) ]: 0.02739853
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
## 0.5182      0.1036
##
## Degrees of Freedom: 3748 Total (i.e. Null); 3747 Residual
## Null Deviance: 4853
## Residual Deviance: 4843 AIC: 4847
## log likelihood: -2421.375
## Nagelkerke R2: 0.003808066
## % pres/err predicted correctly: -1699.416
## % of predictable range [ (model-null)/(1-null) ]: 0.002749748
## *****
## model index: 6

```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.6204
##
## Degrees of Freedom: 3748 Total (i.e. Null);  3748 Residual
## Null Deviance:      4853
## Residual Deviance: 4853  AIC: 4855
## log likelihood:  -2426.565
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1704.104
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##  0.6192998    0.0001413
##
## Degrees of Freedom: 3748 Total (i.e. Null);  3747 Residual
## Null Deviance:      4853
## Residual Deviance: 4853  AIC: 4857
## log likelihood:  -2426.565
## Nagelkerke R2:  1.783372e-08
## % pres/err predicted correctly:  -1704.104
## % of predictable range [ (model-null)/(1-null) ]:  1.283416e-08
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                  CurTask,
                  "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPre	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	4696.875	0.00000	1	1	0.0576406	0.1849374	0.312372	NA	NA	NA	NA
preserved ~ pos	4751.624	54.74912	0	0	0.0382249	-	NA	NA	NA	0.1677105	NA
					0.0117779						
preserved ~ (I(pos^2) + pos)	4753.593	56.71848	0	0	0.0382358	0.0050495	NA	NA	0.0013148	0.1567480	NA
preserved ~ CumErr	4846.751	149.87625	0	0	0.0038080	0.5182255	NA	0.1035782	NA	NA	NA
preserved ~ 1	4855.130	158.25485	0	0	0.0000000	0.6203878	NA	NA	NA	NA	NA
preserved ~ stimlen	4857.130	160.25480	0	0	0.0000000	0.6192998	NA	NA	NA	NA	0.0001413

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.3830 0.3175
##
## Degrees of Freedom: 3223 Total (i.e. Null); 3222 Residual
## Null Deviance: 4023
## Residual Deviance: 3909 AIC: 3913
## log likelihood: -1954.318
## Nagelkerke R2: 0.04876732
## % pres/err predicted correctly: -1345.619
## % of predictable range [ (model-null)/(1-null) ]: 0.03388928
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## -0.07352 -0.02138 0.33459
##
## Degrees of Freedom: 3223 Total (i.e. Null); 3221 Residual
## Null Deviance: 4023
## Residual Deviance: 3935 AIC: 3941
## log likelihood: -1967.739
## Nagelkerke R2: 0.03744661
## % pres/err predicted correctly: -1354.082
## % of predictable range [ (model-null)/(1-null) ]: 0.02781778
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
```

```

## Coefficients:
## (Intercept)          pos
##      0.1896      0.1566
##
## Degrees of Freedom: 3223 Total (i.e. Null);  3222 Residual
## Null Deviance:      4023
## Residual Deviance: 3943  AIC: 3947
## log likelihood:  -1971.422
## Nagelkerke R2:  0.03432362
## % pres/err predicted correctly:  -1358.495
## % of predictable range [ (model-null)/(1-null) ]:  0.02465123
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      0.5916      0.2462
##
## Degrees of Freedom: 3223 Total (i.e. Null);  3222 Residual
## Null Deviance:      4023
## Residual Deviance: 3993  AIC: 3997
## log likelihood:  -1996.391
## Nagelkerke R2:  0.01296174
## % pres/err predicted correctly:  -1379.946
## % of predictable range [ (model-null)/(1-null) ]:  0.009261426
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.7719
##
## Degrees of Freedom: 3223 Total (i.e. Null);  3223 Residual
## Null Deviance:      4023
## Residual Deviance: 4023  AIC: 4025
## log likelihood:  -2011.354
## Nagelkerke R2:  -3.114901e-16
## % pres/err predicted correctly:  -1392.855
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      0.794302      -0.002916

```

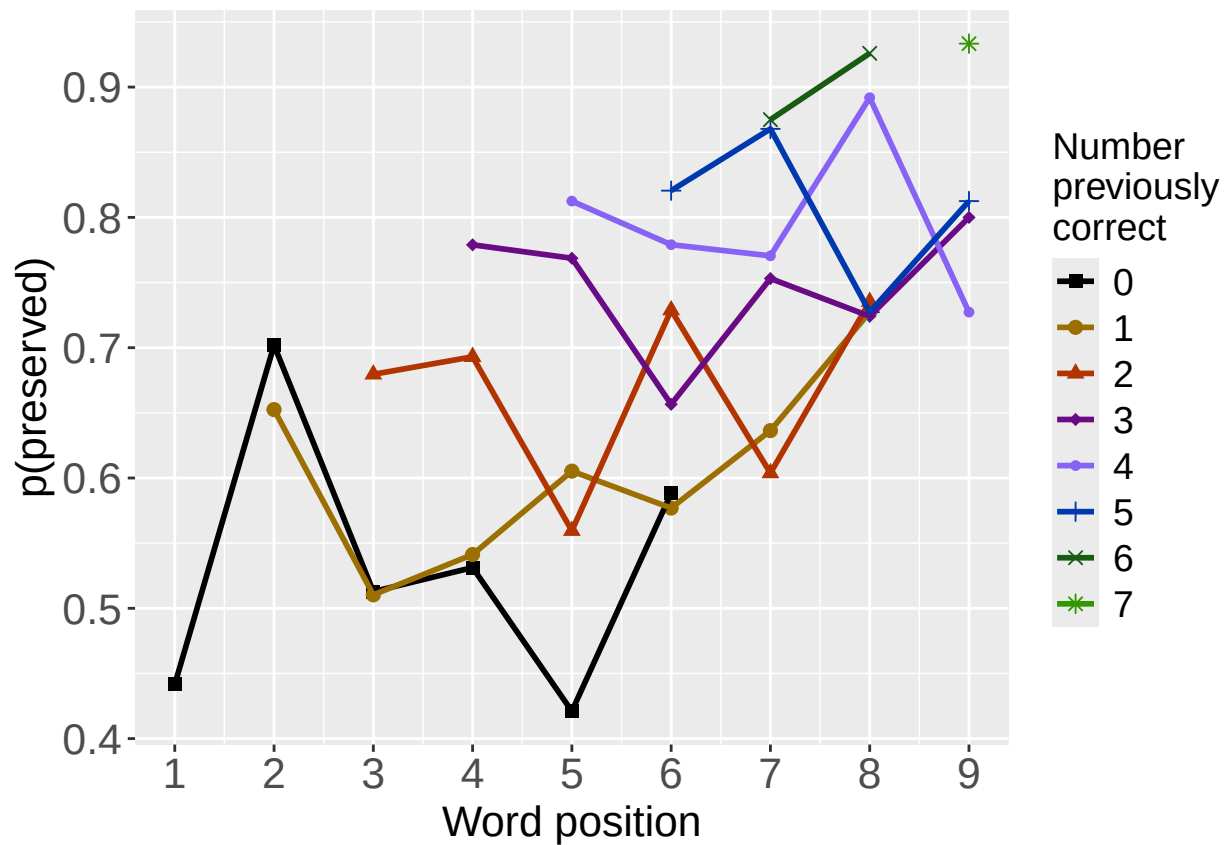
```
##
## Degrees of Freedom: 3223 Total (i.e. Null); 3222 Residual
## Null Deviance: 4023
## Residual Deviance: 4023 AIC: 4027
## log likelihood: -2011.346
## Nagelkerke R2: 7.439606e-06
## % pres/err predicted correctly: -1392.848
## % of predictable range [ (model-null)/(1-null) ]: 5.256864e-06
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	3912.637	0.00000	1e+00	0.9999999	0.0487673	3.3829548	0.3175141	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	3941.472	28.84242	5e-07	0.0000000	0.0374466	-	NA	NA	-	0.3345863	NA
preserved ~ pos	3946.843	34.20817	0e+00	0.0000000	0.0343236	1.895903	NA	NA	NA	0.1565586	NA
preserved ~ CumErr	3996.782	84.14500	0e+00	0.0000000	0.0129617	0.5916231	NA	0.2462343	NA	NA	NA
preserved ~ 1	4024.709	112.07237	0e+00	0.0000000	0.0000000	0.7719058	NA	NA	NA	NA	NA
preserved ~ stimlen	4026.692	114.05527	0e+00	0.0000000	0.0000074	0.7943024	NA	NA	NA	NA	-
											0.0029156

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

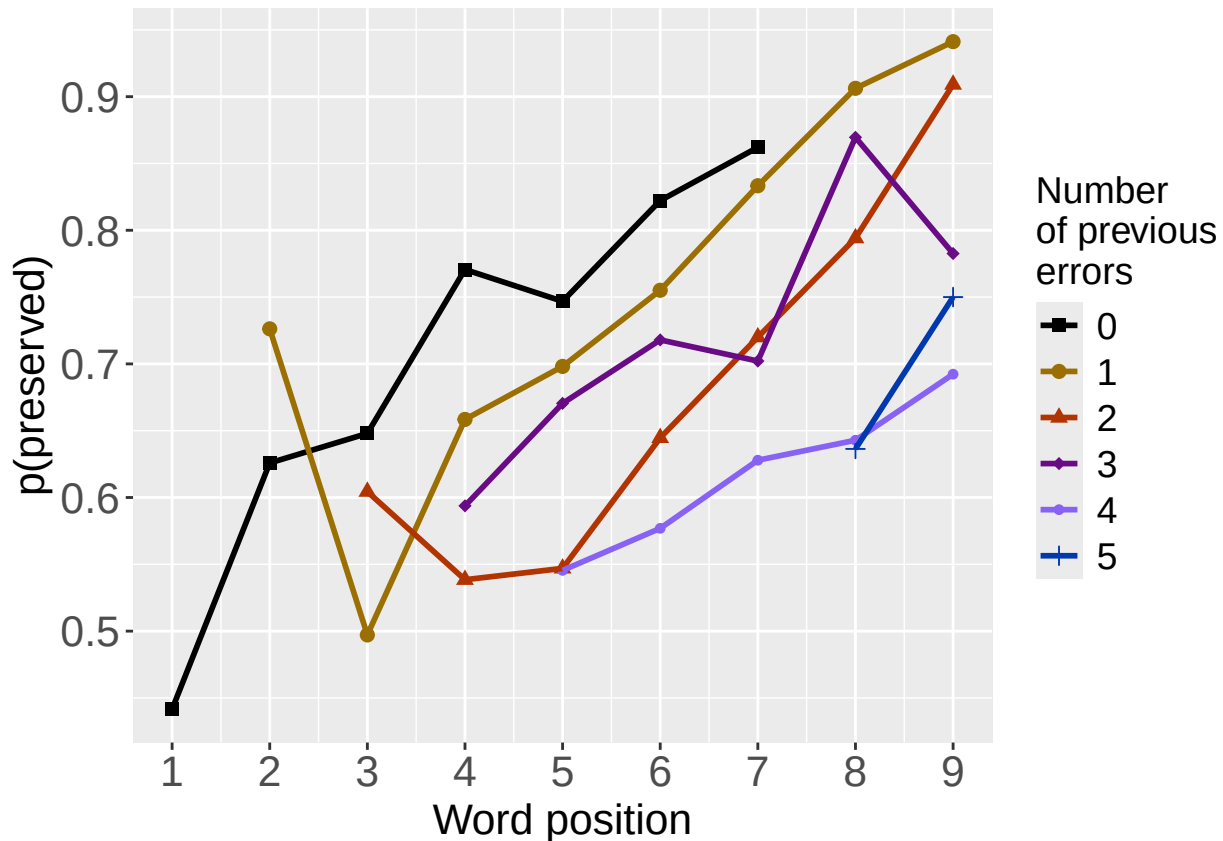
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

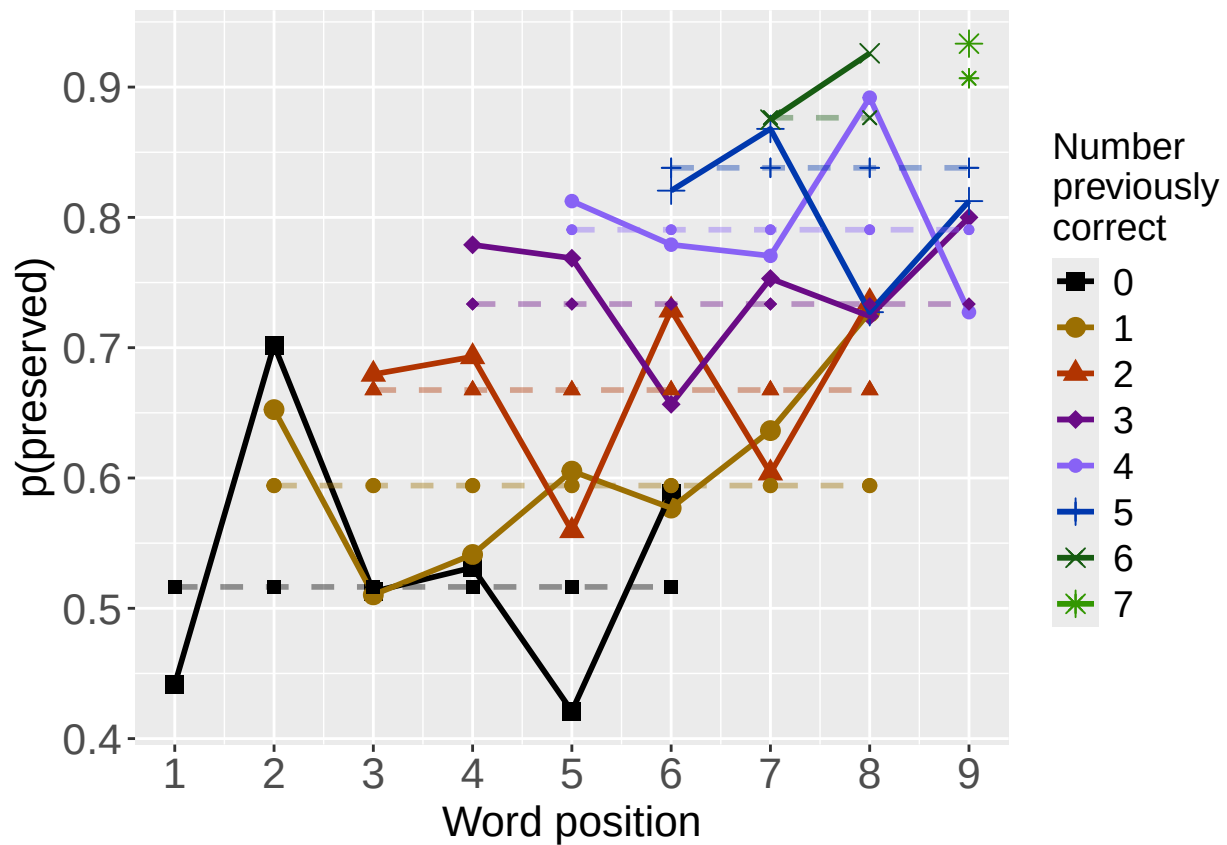
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

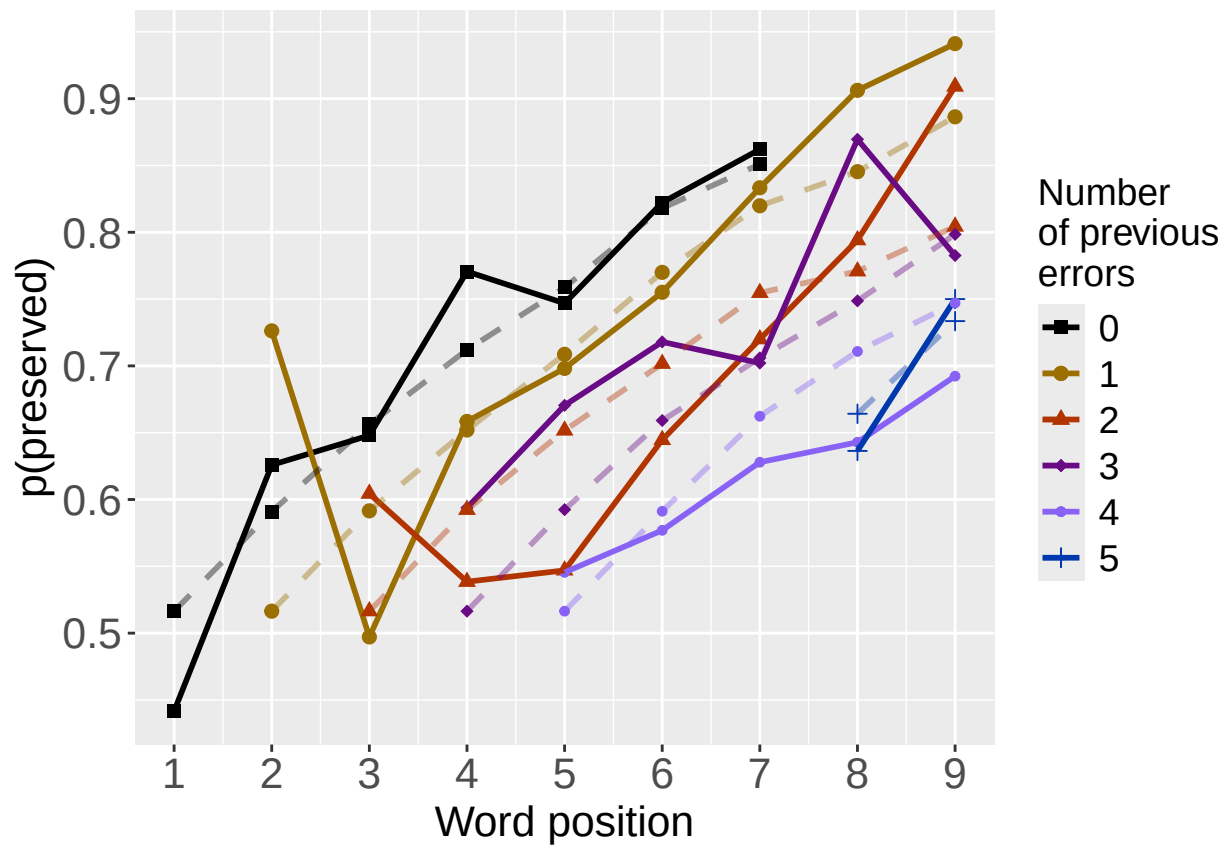
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      0.0657      0.3156
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5129  AIC: 5133
## log likelihood:  -2564.338
## Nagelkerke R2:  0.06183612
## % pres/err predicted correctly:  -1799.382
## % of predictable range [ (model-null)/(1-null) ]:  0.04356838
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)          pos
##   0.0231877    0.2939739   -0.0002108    0.0205561
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5128  AIC: 5136
## log likelihood:  -2564.045
## Nagelkerke R2:   0.06202582
## % pres/err predicted correctly:  -1798.955
## % of predictable range [ (model-null)/(1-null) ]:  0.04379495
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##  -0.1431877   -0.0007766    0.1791957
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5192  AIC: 5198
## log likelihood:  -2596.063
## Nagelkerke R2:   0.04113219
## % pres/err predicted correctly:  -1825.067
## % of predictable range [ (model-null)/(1-null) ]:  0.02992306
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	I(pos^2)	pos
preserved ~ CumPres	5132.675	0.000000	1.0000000	0.8464449	0.0618361	0.0656953	0.3156066	NA	NA
preserved ~ CumPres + I(pos^2) + pos	5136.089	3.413971	0.1814118	0.1535551	0.0620258	0.0231877	0.2939739	-0.0002108	0.0205561
preserved ~ I(pos^2) + pos	5198.125	65.450092	0.0000000	0.0000000	0.0411322	-0.1431877	NA	-0.0007766	0.1791957

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres stimlen
## 0.69805 0.34022 -0.08688
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance: 5315
## Residual Deviance: 5110 AIC: 5116
## log likelihood: -2555.178
## Nagelkerke R2: 0.06775332
## % pres/err predicted correctly: -1790.779
## % of predictable range [ (model-null)/(1-null) ]: 0.04813856
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.0657 0.3156
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance: 5315
## Residual Deviance: 5129 AIC: 5133
## log likelihood: -2564.338
## Nagelkerke R2: 0.06183612
## % pres/err predicted correctly: -1799.382
## % of predictable range [ (model-null)/(1-null) ]: 0.04356838
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      0.61277      -0.01256
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5319
## log likelihood:  -2657.432
## Nagelkerke R2:  0.0001427821
## % pres/err predicted correctly:  -1881.199
## % of predictable range [ (model-null)/(1-null) ]:  0.0001038031
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	stimlen
preserved ~ CumPres + stimlen	5116.356	0.0000	1.0000000	0.9997142	0.0677533	0.6980503	0.3402227	- 0.0868804
preserved ~ CumPres	5132.675	16.3196	0.0002859	0.0002858	0.0618361	0.0656953	0.3156066	NA
preserved ~ stimlen	5318.865	202.5091	0.0000000	0.0000000	0.0001428	0.6127737	NA	- 0.0125643

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      0.0657      0.3156
```

```

##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance: 5315
## Residual Deviance: 5129 AIC: 5133
## log likelihood: -2564.338
## Nagelkerke R2: 0.06183612
## % pres/err predicted correctly: -1799.382
## % of predictable range [ (model-null)/(1-null) ]: 0.04356838
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres pos
## 0.02591 0.29401 0.01878
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance: 5315
## Residual Deviance: 5128 AIC: 5134
## log likelihood: -2564.045
## Nagelkerke R2: 0.06202555
## % pres/err predicted correctly: -1798.966
## % of predictable range [ (model-null)/(1-null) ]: 0.04378897
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## -0.1332 0.1727
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance: 5315
## Residual Deviance: 5192 AIC: 5196
## log likelihood: -2596.069
## Nagelkerke R2: 0.04112834
## % pres/err predicted correctly: -1825.117
## % of predictable range [ (model-null)/(1-null) ]: 0.02989662
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	pos
preserved ~ CumPres	5132.675	0.000000	1.0000000	0.6698278	0.0618361	0.0656953	0.3156066	NA
preserved ~ CumPres + pos	5134.090	1.414812	0.4929211	0.3301722	0.0620256	0.0259064	0.2940134	0.0187788
preserved ~ pos	5196.137	63.461812	0.0000000	0.0000000	0.0411283	- 0.1331636	NA	0.1726978


```

CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                             RunLabel,"_",CurTask,
                             "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPred(pos^2)	pos	stimlen
preserved ~ CumPres + stimlen	5116.356	0.000000	1.000000	0.999714	0.067753	0.698050	0.3402227	NA	- 0.0868804
preserved ~ CumPres	5132.675	0.000000	1.000000	0.846444	0.061836	0.065695	0.3156066	NA	NA
preserved ~ CumPres	5132.675	16.319597	0.000285	0.000285	0.061836	0.065695	0.3156066	NA	NA
preserved ~ CumPres	5132.675	0.000000	1.000000	0.669827	0.061836	0.065695	0.3156066	NA	NA
preserved ~ CumPres + pos	5134.091	0.414812	0.492921	0.330172	0.062025	0.025906	0.40134	NA	0.0187788
preserved ~ CumPres + I(pos^2) + pos	5136.089	3.413971	0.181411	0.153555	0.062025	0.023187	0.293739	-	0.0205561
preserved ~ pos	5196.137	3.461812	0.000000	0.000000	0.0411283	-	NA	NA	0.1726978
					0.1331636				NA
preserved ~ I(pos^2) + pos	5198.125	5.450092	0.000000	0.000000	0.0411322	-	NA	-	0.1791957
preserved ~ stimlen	5318.865	202.509075	0.000000	0.000000	0.0001428	0.6127737	NA	NA	NA
									- 0.0125643

```

# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

## *****
## model index: 3

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      stimlen      log_freq
##      0.60254      0.33940     -0.07423      0.03882
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5106  AIC: 5114
## log likelihood:  -2552.901
## Nagelkerke R2:  0.06921976
## % pres/err predicted correctly:  -1788.785
## % of predictable range [ (model-null)/(1-null) ]:  0.04919783
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      stimlen
##      0.69805      0.34022     -0.08688
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5110  AIC: 5116
## log likelihood:  -2555.178
## Nagelkerke R2:  0.06775332
## % pres/err predicted correctly:  -1790.779
## % of predictable range [ (model-null)/(1-null) ]:  0.04813856
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.516
##
## Degrees of Freedom: 4020 Total (i.e. Null);  4020 Residual
## Null Deviance:      5315
## Residual Deviance: 5315  AIC: 5317
## log likelihood:  -2657.643
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1881.394
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	stimlen	log_freq
preserved ~ CumPres + stimlen + log_freq	5113.8030	0.000000	1.000000	0.781855	0.069219	0.6025432	0.3393982	-	0.0388211
preserved ~ CumPres + stimlen	5116.3562	0.553025	0.279008	0.218144	0.067753	0.6980503	0.3402227	-	NA
preserved ~ 1	5317.2862	0.483167	0.000000	0.000000	0.000000	0.5160018	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumPres + stimlen + log_freq
##      Df Deviance   AIC
## CumPres  1   5309.1 5315.1
## stimlen  1   5118.1 5124.1
## log_freq  1   5110.4 5116.4
## <none>    5105.8 5113.8

```

```

#####
# Single deletions from best model
#####

```

```

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),

```

```

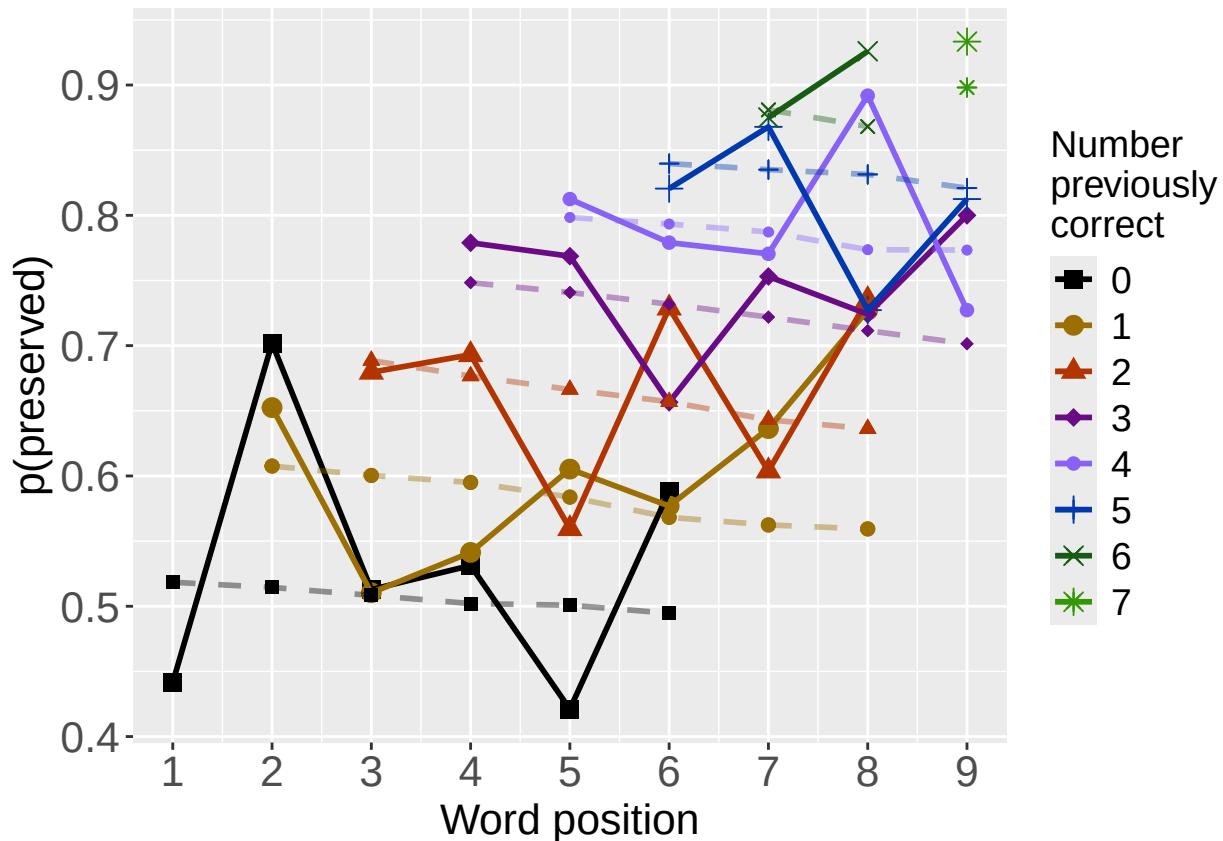
row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```

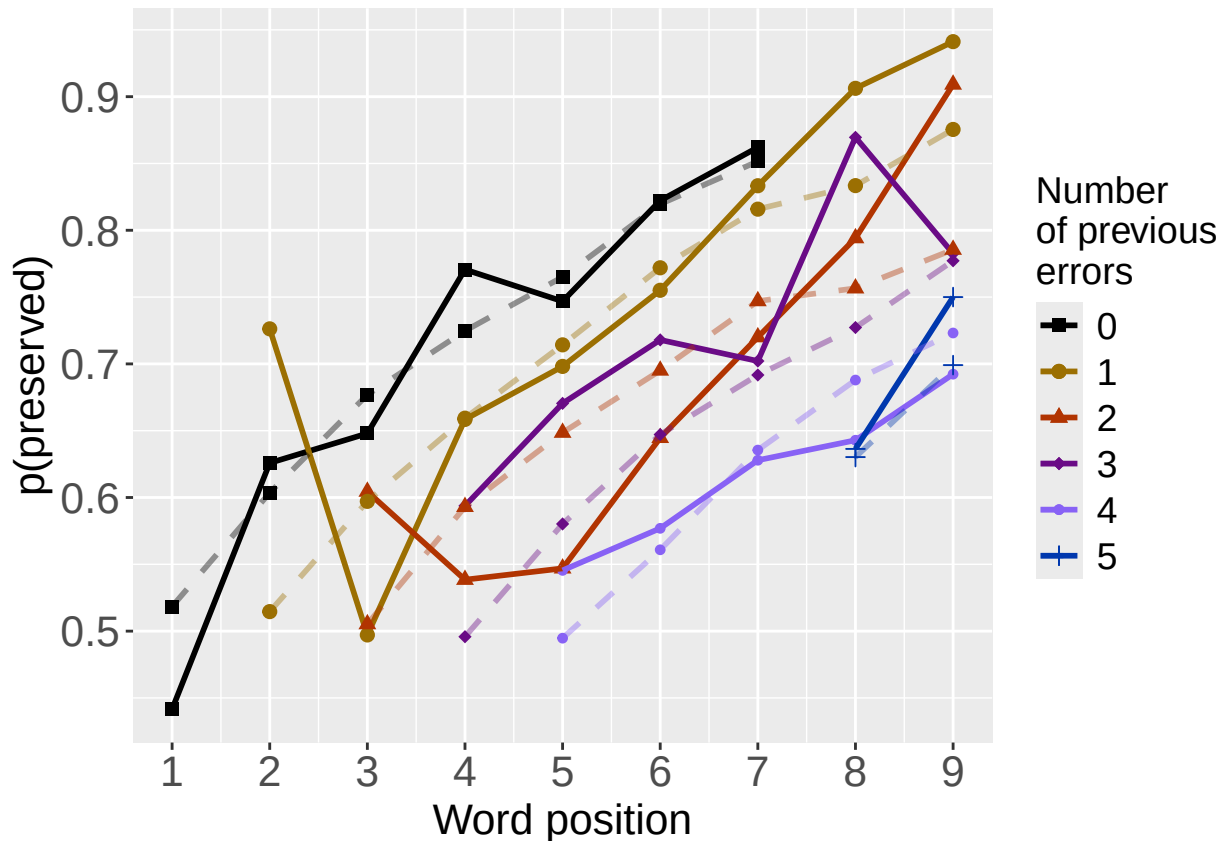


```

PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)

```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      0.0657      0.3156
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4019 Residual
## Null Deviance:      5315
## Residual Deviance: 5129 AIC: 5133
## log likelihood: -2564.338
## Nagelkerke R2: 0.06183612
## % pres/err predicted correctly: -1799.382
## % of predictable range [ (model-null)/(1-null) ]: 0.04356838

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      stimlen
##      0.69805      0.34022     -0.08688
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4018 Residual
## Null Deviance:      5315
## Residual Deviance: 5110 AIC: 5116
## log likelihood: -2555.178
## Nagelkerke R2: 0.06775332
## % pres/err predicted correctly: -1790.779
## % of predictable range [ (model-null)/(1-null) ]: 0.04813856
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      stimlen      log_freq
##      0.60254      0.33940     -0.07423      0.03882
##
## Degrees of Freedom: 4020 Total (i.e. Null); 4017 Residual
## Null Deviance:      5315
## Residual Deviance: 5106 AIC: 5114
## log likelihood: -2552.901
## Nagelkerke R2: 0.06921976
## % pres/err predicted correctly: -1788.785
## % of predictable range [ (model-null)/(1-null) ]: 0.04919783
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

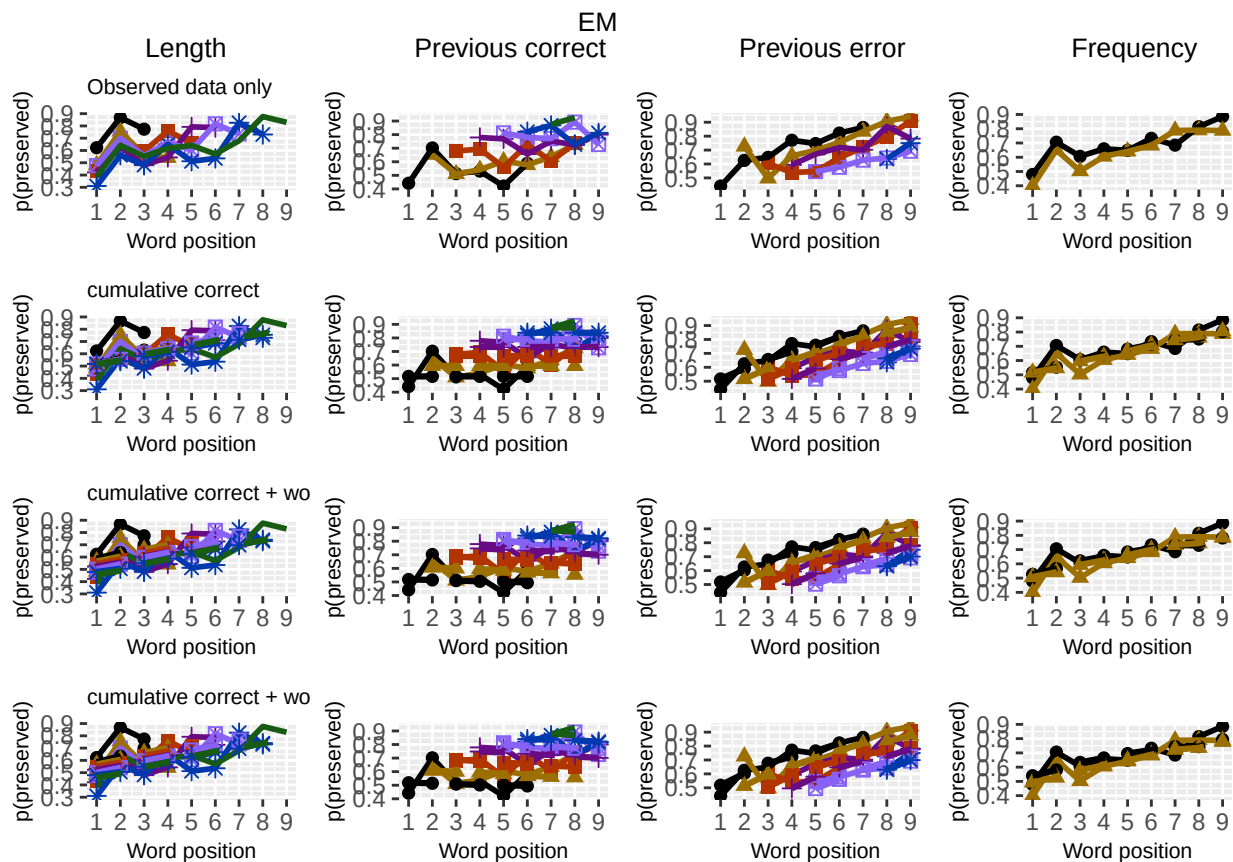
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 8 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```

```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumPres	stimlen	log_freq
McFadden	0.0368508	0.0013708	0.0011899
SquaredCorrelation	0.0475048	0.0017258	0.0015329
Nagelkerke	0.0475048	0.0017258	0.0015329
Estrella	0.0484108	0.0017902	0.0015628

	deviance	deviance_explained
CumPres + stimlen + log_freq	5105.803	209.4832
CumPres + stimlen	5110.356	204.9301
CumPres	5128.675	186.6105
null	5315.286	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumPres + stimlen + log_freq	3.941146	100.00000	2.173456
CumPres + stimlen	3.855487	97.82654	8.745140
CumPres	3.510828	89.08140	89.081404
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumPres + stimlen + log_freq CumPres + stimlen + log_freq 5105.803          209.4832          3.941146          100.00000
## CumPres + stimlen              CumPres + stimlen 5110.356          204.9301          3.855487          97.82654
## CumPres                        CumPres 5128.675          186.6105          3.510828          89.08140
## null                            null 5315.286           0.0000          0.000000          NA
##                                increment_in_explained
## CumPres + stimlen + log_freq          2.173456
## CumPres + stimlen                    8.745140
## CumPres                             89.081404
## null                               0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumPres  0.93580733
## stimlen  0.03399611
## log_freq 0.03019656

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumPres	0.5931549	5128.675
preserved ~ CumPres+stimlen+log_freq	0.5981418	5105.803
preserved ~ CumPres+stimlen	0.6004682	5110.356

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)

sse_table
```

```
##
## 1          preserved ~ CumPres      0.5931549      5128.675      0.00000000      -0.004986930      -0.007313390
## 2 preserved ~ CumPres+stimlen+log_freq 0.5981418      5105.803      0.00498693      0.000000000      -0.002326459
## 3          preserved ~ CumPres+stimlen 0.6004682      5110.356      0.00731339      0.002326459      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumPres	diff_CumPres+stimlen+log_freq	diff_CumPres+stimlen
preserved ~ CumPres	0.0000000	-0.0049869	-0.0073134
preserved ~ CumPres+stimlen+log_freq	0.0049869	0.0000000	-0.0023265
preserved ~ CumPres+stimlen	0.0073134	0.0023265	0.0000000